



EDINBURGH NAPIER UNIVERSITY

SET081010/SET08702 - MultiAgent Systems

Lab 01 - Agency

Dr Simon Wells

1 Aims

At the end of the practical portion of this topic you will:

- Have a good idea of how to structure code to implement a basic Agent
- Have some experience of using Python & asyncio to create co-routines to allow your agent and environment to exist independently of each other within the same program.
- Have adapted the basic agent framework to satisfy a different problem.
- Have explored some agent frameworks to contrast our “Agents from scratch” approach with more “batteries included” approaches.

NOTICE: These exercises are meant to be a starting point. They are meant to show you a direction that you can head in, but it’s down to you to explore that path. If you discover something interesting, then consider sharing it with the class. It’s good to go down the rabbithole sometimes.

2 Activities

2.1 A Thermostat Agent

A simple agent to operate like a Thermostat is a type of purely reactive agent like this:

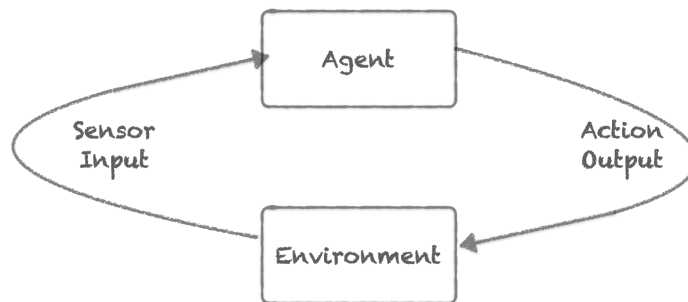


Figure 1

Here’s a basic implementation of a simple Thermostat agent. It’s not full functional for two reasons:

1. to give you something to do
2. because when is a program every complete (especially if you can come up with other things to do with it).

Implement the following code and get it running. You will need Python 3 installed on your machine and for it to be running correctly.

```
1 #!/usr/bin/env python3
2
3 import asyncio
4 import random
5
6 temp = 20
7
8 class Agent():
```

```

9 |         async def run(self):
10 |             while True:
11 |                 if (temp == 20):
12 |                     print("Temp is good")
13 |                 else:
14 |                     print("Temp is bad")
15 |                     await asyncio.sleep(1)
16 |
17 | async def environment():
18 |     while True:
19 |         global temp
20 |         temp = random.randrange(12,28)
21 |         print(temp)
22 |         await asyncio.sleep(1)
23 |
24 | async def main():
25 |     agent = Agent()
26 |     await asyncio.gather(
27 |         environment(),
28 |         agent.run(),
29 |     )
30 |
31 | if __name__ == "__main__":
32 |     asyncio.run(main())

```

Notice that we have an Agent class which uses the `async` keyword on the `run` function to enable run to proceed asynchronously. We also use a `while True` to get an infinite loop so our Agent is immortal and runs forever. Within that loop it has a simple behaviour, check the temperate and do something, then sleep for one second before running again.

Our environment is not even a class in this example¹, just a simple function that also runs asynchronously to randomly set a global value, “temp” that represents the temperature the also sleeps for one second.

Our

Modify so that:

- agent turns heater on when temp below 20 and of when above 20. The heater could be represented by a global boolean variable however rather than an actual value. For eaxmple, True when on, False when off.
- environment increases temp by `rand(x)` if heater on
- environment decreases temp by `rand(x)` if heater off
- Simulates temp rising and falling at different rates due to weather

There are lots of ways you could extend this, perhaps turning by encapsulating the environment into it’s own class, a bit like Agent. Different strategies for maintaining the temperature for example, or learning from the change in temperature, perhaps with an agent representing the occupant of the heated space, when they arrive and leave. This would possibly need you to represent time explicitly, but would open the way to putting some actual learning within the agent to learn what it’s owner liked so it could pre-emptively set the temperature. This would take use away from a purely reactive agent towards one that was goal-driven and deliberative.

2.2 Hoover Agent

Make a copy of the basic Thermostat agent example into a new `hoover.py` file. Now you are going to implement a simple agent that will operate like a robotic hoover, moving from location to location, and vacuuming dirt if the location is dirty.

¹We’ll elaborate our environment modelling later in the “Environment” topic but YAGNY for now.

Make whatever changes you need to implement the following²

- 2 locations [left — right]
- location can be clean or dirty
- agent can move left or right
- if location is dirty then hoover should vacuum
- Environment should randomly set left or right to dirty over time

Now start considering some elaborations of your own design, perhaps more locations, of a randomly generate size, which would open up the opportunity for the agent to explore the area to clean, like a Roomba. Again, the sky is the limit but the main idea is to get you exploring your own ideas for how to turn simple agent scenarios into code.

2.3 Deeper Dive

The Python asyncio library is a really useful Python library that you can use to create software with parts that functions concurrently and asynchronously. This is what we're going to use, for the most part, to get the agents that we create to operate independently of each other.

A good place to start is the python documentation for asyncio: <https://docs.python.org/3/library/asyncio-task.html> and for coroutines: <https://docs.python.org/3/glossary.html#term-coroutine>

There is a lot of information about coroutines and the asyncio library online so a good way to get to grips with it is to search for a few tutorials and play with the code, then to fold back your learning from those explorations into your own agent code.

There's not really a wrong way to go about this, just code that works and code that doesn't; code that satisfies its design goals, and code that doesn't, code that is elegant, and code that isn't; and importantly, code that fulfills its goals in a reasonable amount of time, and code that doesn't.

2.4 Reimplementation

There are a lot of “batteries included” agent frameworks. This means that they include lots of functionality for you, and have already implemented much of the basic stuff, like creating agents that operate asynchronously. However I think that that misses an important aspect of the learning, and skill development, process – being able to take the basic idea and confidently implement your own from scratch. Building from scratch affords deep understanding (even if you don't really build everything from scratch in the real world). This is similar to learning about Algorithms and Data Structures, you implement the basic algorithms and data structures from scratch so that you get a thorough grounding in how they work, then for real applications, you use a standard library.

Two places to start, for comparison are AgentPy³ and the Python Agent DEvelopment Framework (PADE)⁴. Both are basically Python libs that allow you to create agents, that are managed for you, then to use those agents to complete tasks and perform behaviours.

Similarly, the grand-daddy of agent development is the Java Agent Development Environment (JADE)⁵ which is built around Java.

Explore using some different agent frameworks to reimplement first your Thermostat agent and then your Hoover agent. Note how things are done as you go along. This might give you ideas for how to structure your own approaches when building from scratch in this module.

²You might need to do some reading of API and library documentation to find out about additional functionality of Python and its libraries. This is fine. It's part of the learning process

³<https://agentpy.readthedocs.io/>

⁴<https://pade.readthedocs.io/en/latest/index.html>

⁵<https://jade.tilab.com/>